

Assignment No. 2

Q.1 Explain MongoDB CRUD Operations with an example.

In MongoDB, CRUD operations are performed on documents that are stored inside collections. CRUD stands for Create, Read, Update, and Delete. These operations represent the four basic functions used to interact with a database. MongoDB is a NoSQL database, which means it stores data in a flexible JSON-like format called BSON (Binary JSON) rather than in tables and rows like traditional relational databases.

CRUD operations allow users and applications to add new data, retrieve existing data, modify stored data, and remove unwanted data from the database. These operations are essential for building dynamic applications such as web applications, mobile applications, and enterprise systems.

1. Create (Insert Data)

- Used to add new documents to a collection.
- Methods: insertOne() and insertMany()

Example:

```
db.students.insertOne({name: "Rahul", age: 20, course: "IT"})
```

```
db.students.insertMany([
  {name: "Tejsi", age: 22, course: "CS"},
  {name: "Janhavi", age: 22, course: "IT"}
])
```

2. Read (Retrieve Data)

- Used to fetch data from the database.
- Methods: find() and findOne()

Example:

```
db.students.find() → Displays
```

```
all records
```

```
db.students.find({course: "IT"})
```

```
→ Displays only IT students
```

3. Update (Modify Data)

MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
Permanently Affiliated to University of Mumbai
Department of Information Technology
NBA Accredited

TE-Sem VI Course: Web Lab(ITL602) Academic Year: 2025-26 Roll No.: 607

- Used to update existing records.
- Methods: updateOne() and updateMany()

Example:

```
db.students.updateOne(
  {name: "Ari"},
  {$set: {age: 21}}
)

db.students.updateMany(
  {course: "IT"},
  {$set: {course: "Information Technology"}}
)
```

4. Delete (Remove Data)

- Used to delete documents.
- Methods: deleteOne() and deleteMany()

Example:

```
db.students.deleteOne({name: "Ari"}) db.students.deleteMany({course:
"CS"})
```

In this example, MongoDB deletes the document from the students collection where the name field is Ari. If multiple documents match the condition and the user wants to remove all of them, the deleteMany() method can be used.

Q.2 Explain URL building in Python Flask.

URL building in Flask refers to the process of dynamically generating URLs for different routes in a web application. Instead of manually writing URLs in the code, Flask provides a function called url_for() that automatically creates the correct URL for a specific view function. This makes the application easier to maintain and reduces errors caused by hardcoding URLs.

In Flask, every route is associated with a view function. The url_for() function uses the name of the view function to generate the corresponding URL. If the URL structure changes in the future, the developer only needs to update the route definition, and all generated URLs will automatically update.

MCT
MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
Permanently Affiliated to University of Mumbai
Department of Information Technology
NBA Accredited

TE-Sem VI Course: Web Lab(ITL602) Academic Year: 2025-26 Roll No.: 607

The `url_for()` function is used to build a URL for a specific function. It takes the name of the function as an argument and returns the correct URL.

Syntax:

```
url_for('function_name', **values)
```

- `function_name` → Name of the view function.
- `values` → Optional parameters that are passed to the route.

Example of URL Building in Flask :

```
from flask import Flask, url_for  
  
app = Flask(__name__)  
  
@app.route('/') def home():  
    return "Welcome to Home Page"  
  
@app.route('/about') def  
about():  
    return "About Page" with  
app.test_request_context():  
print(url_for('home')) print(url_for('about'))
```

```
PS C:\Users\Pranali\Desktop\Vrushali\webx> python app.py  
/  
/about
```

In this example, the `url_for()` function generates URLs for the home and about routes based on their function names.

Example with Dynamic Parameters

Flask also allows passing parameters in the URL.

```
from flask import Flask, url_for
```

```
app = Flask(__name__)  
@app.route('/user/<username>') def  
profile(username):  
    return f"User: {username}"
```

with app.test_request_context():
print(url_for('profile', username='Rahul')) Output:

```
PS C:\Users\Pranali\Desktop\Vrushali\webx> python app.py  
/user/Rahul
```

Here, username is passed as a parameter to the profile function, and url_for() builds the URL dynamically.

URL building is important in Flask for several reasons:

1. Avoids hard-coded URLs
Developers do not need to manually write the URL paths.
2. Easy maintenance
If the route changes, the URL automatically updates everywhere in the application.
3. Supports dynamic values
It allows passing parameters in the URL.
4. Improves application structure
It keeps the application organized and scalable.

Q.3 What is the XML Http Request object in AJAX? Explain its methods and properties.

The XMLHttpRequest (XHR) object is an important component used in AJAX (Asynchronous JavaScript and XML) for communication between a web browser and a web server. It allows data to be exchanged between the client and the server without reloading the entire webpage. This makes web applications faster and more interactive.

The XMLHttpRequest object is mainly used to send requests to the server and receive responses asynchronously. Although the name suggests XML, it can handle different types of data such as JSON, HTML, text, and XML.

Working of XMLHttpRequest Object

The working of the XMLHttpRequest object involves the following steps:

1. A new XMLHttpRequest object is created.
2. The request is initialized using the open() method.
3. The request is sent to the server using the send() method.
4. The server processes the request and sends a response.
5. The response is received and used to update the webpage dynamically.

Methods of XMLHttpRequest

The XMLHttpRequest object provides several methods to handle server communication.

1. open()

The open() method is used to initialize a request. It specifies the HTTP method, the URL, and whether the request is asynchronous.

Syntax:

```
xhr.open(method, url, async);
```

2. send()

The send() method is used to send the request to the server. **Example:**

```
xhr.send();
```

If data is to be sent (like in POST request), it can be passed as a parameter.

3. setRequestHeader()

This method is used to set HTTP headers before sending the request.

Example:

```
xhr.setRequestHeader("Content-Type", "application/json");
```

4. abort()

The abort() method is used to cancel the request if it is no longer needed. **Example:**

```
xhr.abort();
```

5. getResponseHeader()

This method is used to get a specific response header from the server.

Example:

```
xhr.getResponseHeader("Content-Type");
```

Properties of XMLHttpRequest

The XMLHttpRequest object contains several properties that provide information about the request and response.

1. readyState

This property indicates the current state of the request.

Value	State	Description
0	UNSENT	Request not initialized
1	OPENED	Connection established

MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
Permanently Affiliated to University of Mumbai
Department of Information Technology
NBA Accredited

TE-Sem VI Course: Web Lab(ITL602) Academic Year: 2025-26 Roll No.: 607

2	HEADERS_RECEIVED	Request received by server
3	LOADING	Processing request
4	DONE	Request completed

2. status

This property returns the HTTP status code.

Examples:

- 200 → Success
- 404 → Not Found
- 500 → Server Error

3. statusText

This property returns the status message.

Example:

xhr.statusText Output:
"OK"

4. responseText

This property contains the response data in text format.

Example:

xhr.responseText

5. responseXML

This property returns the response in XML format if the server sends XML data.

Example:

xhr.responseXML

Q.4 Develop a Django app that stores and displays employee records from the database.

Django is a Python web framework that follows the Model-View-Template (MVT) architecture. In this application, employee records are stored and managed using a SQLite database. The model is used to define the structure of employee data with fields such as name, email, department, and salary. Django ORM is used to interact with the database, making it easy to insert, retrieve, update, and delete records without writing SQL queries.

The application includes view functions to handle different operations such as displaying all employee records, adding new records, updating existing data, and deleting records. The home page displays the employee details in a table format, while a form is used for adding

MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
Permanently Affiliated to University of Mumbai
Department of Information Technology
NBA Accredited

TE-Sem VI Course: Web Lab(ITL602) Academic Year: 2025-26 Roll No.: 607

and editing data. URL routing is used to connect these functions, and the Django development server is used to run the application and view the output in a web browser.

Code:

```
from django.conf import settings

settings.configure(
    DEBUG=True,

    'django.contrib.auth'
],
DATABASES={
    'default': {
'ENGINE':
'django.db.backends.sqlite3',
        'NAME': 'employee_db.sqlite3'
    }
}
)

import django
django.setup()

from django.db import models, connection
from django.http import HttpResponse
from django.urls import path
from django.shortcuts import redirect

# ----- MODEL ----- class
Employee(models.Model):    name =
models.CharField(max_length=100)    email =
models.EmailField()
    department
                                =
models.CharField(max_length=100)
    salary = models.IntegerField()

class Meta:
    app_label = 'employees'

# ----- CREATE TABLE -----

SECRET_KEY='abc123',
ROOT_URLCONF=__name__,
INSTALLED_APPS=[
    'django.contrib.contenttypes',

- try:    with connection.schema_editor() as
schema:    schema.create_model(Employee)
except:    pass

# ----- DEFAULT DATA -----
-
if not Employee.objects.exists():
Employee.objects.create(name="Kartiki",
email="kartiki@gmail.com",
department="HR", salary=45000)
Employee.objects.create(name="Ankit",
email="ankit@xyz.com",
department="Finance", salary=60000)

# ----- HOME PAGE -----
def home(request):
    q = request.GET.get('q', "")
    employees
                                =
Employee.objects.filter(name__icontains=q) if
q else Employee.objects.all()

    rows = ""    for e
in employees:
rows += f""
        <tr>
            <td>{e.name}</td>
            <td>{e.email}</td>
            <td>{e.department}</td>
            <td>₹{e.salary}</td>
            <td>
                <a href='/form/{e.id}'>Edit</a> |
                <a href='/delete/{e.id}'>Delete</a>
```


MCT
MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
Permanently Affiliated to University of Mumbai
Department of Information Technology
NBA Accredited

TE-Sem VI Course: Web Lab(ITL602) Academic Year: 2025-26 Roll No.: 607

```

    </td>
</tr>
""""

return HttpResponse(f""""
<body                                style='font-
family:Arial;background:lineargradient(135deg
,#667eea,#764ba2);'>
    <div
style='width:80%;margin:auto;background:whi
te;padding:25px;                margin-
top:40px;border-
radius:12px;box-shadow:0        5px        15px
rgba(0,0,0,0.2)'>

        <h1    style='text-align:center'>Employee
Dashboard</h1>

        <form style='text-align:center'>
<input name='q' placeholder='Search...'
value='{q}' style='padding:8px;width:250px'>
<button        style='padding:8px
15px;background:#667eea;color:white;border:
none;border-radius:5px'>
            Search
        </button>
    </form>

    <br>
    <a href='/form'
style='color:#333;fontweight:bold'> Add
Employee</a>

    <table border='1' width='100%'
cellpadding='10' style='margin-
top:15px;border-collapse:collapse'>
        <tr style='background:#eee'>
            <th>Name</th>
<th>Email</th>
            <th>Dept</th>
            <th>Salary</th>
            <th>Action</th>

```

```

    </tr>
    {rows}
</table>
</div>
</body>
""""

# ----- ADD / EDIT FORM -----
----
def form(request, id=None):
    emp = Employee.objects.filter(id=id).first()

    if request.method == "POST":
    if emp:
        emp.name = request.POST['name']
        emp.email = request.POST['email']
        emp.department =
request.POST['department']        emp.salary
= request.POST['salary']
        emp.save()
    else:

Employee.objects.create(**request.POST)

    return redirect('/')

return HttpResponse(f""""
<body
style='fontfamily:Arial;background:lineargradi
ent(135deg,#764ba2,#667eea);'>
    <div
style='width:50%;margin:auto;background:whi
te;padding:25px;
        margin-top:60px;border-
radius:12px;box-shadow:0        5px        15px
rgba(0,0,0,0.2)'>

        <h2 style='text-align:center'>{'Edit' if emp
else 'Add'} Employee</h2>

```


MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
Permanently Affiliated to University of Mumbai
Department of Information Technology
NBA Accredited

TE-Sem VI Course: Web Lab(ITL602) Academic Year: 2025-26 Roll No.: 607

```
<form method='POST'
style='display:grid;grid-templatecolumns:1fr
1fr;gap:15px'>

    <input                name='name'
placeholder='Name'
value='{emp.name if emp else ""}'
required>

    <input                name='email'
placeholder='Email'
value='{emp.email if emp else ""}'
required>

    <input name='department'
placeholder='Department'
value='{emp.department if emp else ""}'
required>

    <input                name='salary'
placeholder='Salary'
value='{emp.salary if emp else ""}'
required>

    <button                style='grid-column:span
2;padding:10px;
background:#667eea;color:white;
border:none;borderradius:6px'>
        Save
    </button>
</form>
</div>
</body>
""")

# ----- DELETE ----- def
delete(request, id):
    Employee.objects.filter(id=id).delete()

    return redirect('/')

]

# ----- URLS -----
urlpatterns = [
    path("", home),
    path('form', form),
    path('form/<int:id>', form),
    path('delete/<int:id>', delete),

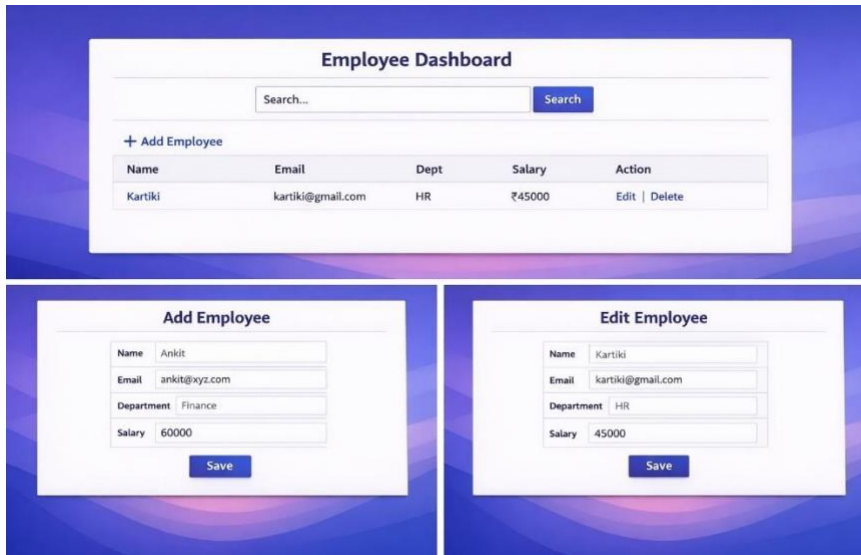
# ----- RUN SERVER -----
from django.core.management import
execute_from_command_line

execute_from_command_line(["manage.py",
"runserver", "0.0.0.0:8000"])
```

Output:

 MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
 Permanently Affiliated to University of Mumbai
Department of Information Technology
 NBA Accredited

TE-Sem VI Course: Web Lab(ITL602) Academic Year: 2025-26 Roll No.: 607



Q.5 Develop a Flask-based To-Do List Application where users can add and delete tasks.

Code:

```
from flask import Flask, request, redirect
```

```
app = Flask(__name__)
```

```
# Store tasks
tasks = ["Buy groceries",
         "Complete the report", "Gym workout",
         "Read a book"]
```

```
@app.route("/", methods=["GET", "POST"])
def home():
    if request.method == "POST":
        task = request.form.get("task")
```

```
        if task:
            tasks.append(task)
        return redirect("/")
```

```
html = """
<html>
<head>
    <title>To-Do App</title>
    <style>
        body {
            font-family: Arial;
```

```
            background: linear-gradient(to right,
            #36d1dc, #5b86e5);
        }
        .container {
            width: 50%;
            margin:
            auto;
            background:
            white;
            padding: 25px;
            margin-top: 60px;
```

```
            border-radius: 12px;
            box-shadow: 0 5px 15px
            rgba(0,0,0,0.2);
        }
        h2 {
            text-align: center;
        }
        .input-box {
            display: flex;
            gap: 10px;
            padding: 10px;
```

MCT
MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
Permanently Affiliated to University of Mumbai
Department of Information Technology
NBA Accredited

TE-Sem VI Course: Web Lab(ITL602) Academic Year: 2025-26 Roll No.: 607

```
width: 70%;          border-
radius: 5px;         border:
1px solid #ccc;
    }               button {
padding: 10px 15px;
background: green;
color: white;
border: none;
border-radius: 5px;
cursor: pointer;
    }               .task {
display: flex;          justify-content:
space-between;          margin-top:
10px;                  padding: 10px;
background: #f1f1f1;    border-
radius: 6px;
    }               a {
color: red;
text-decoration:
none;
font-weight: bold;
    }
</style>
</head>

<body>

<div class="container">
    <h2> To-Do List</h2>

    <form method="POST"
class="inputbox">
```

```
        <input type="text" name="task"
placeholder="Enter a new task..." required>
        <button>Add Task</button>
    </form>
    """

    for i, t in enumerate(tasks):
html += f"""    <div
class="task">
        <span>{t}</span>
        <a href="/delete/{i}"> </a>
    </div>
    """

html += """
</div>

</body>
</html>
"""

return html

@app.route("/delete/<int:index>")
def delete(index):    if index <
len(tasks):        tasks.pop(index)
return redirect("/")

if __name__ == "__main__":
    app.run(debug=True)
```

Output:

MCT
MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI
Permanently Affiliated to University of Mumbai
Department of Information Technology
NBA Accredited

TE-Sem VI Course: Web Lab(ITL602) Academic Year: 2025-26 Roll No.: 607

 To-Do List

Add Task

Assignment 2 (webx)



Ready file of BI



Assignment 2 (sensor)

